

Homework - Neural networks - Part B (55 points)

Gradient descent for simple two and three layer models

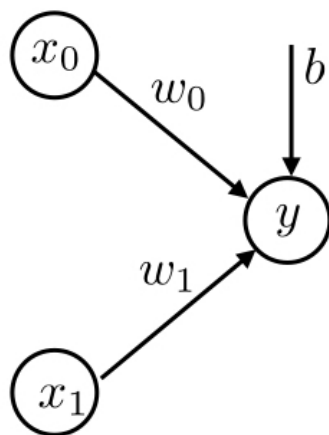
by Brenden Lake

The first part of this assignment implements the gradient descent algorithm for a simple artificial neuron. The second part implements backpropagation for a simple network with one hidden unit. Remember that we strongly suggest you *TURN OFF* any autocompletion functionality (e.g., in VSCode and Cursor) so that you can learn and think as you do this assignment. For instance, in VSCode, edit `settings.json` (on mac search via command-P) such that `quickSuggestions` are off.

```
"editor.quickSuggestions": {
  "other": "off",
  "comments": "off",
  "strings": "off"
}
```

Alternatively, working with Jupyter notebooks in the terminal also removes this temptation.

In the first part, the neuron will learn to compute logical OR. The neuron model and logical OR are shown below, for inputs x_0 and x_1 and target output y .



logical OR

x_0	x_1	y
0	0	0
0	1	1
1	0	1
1	1	1

This assignment requires some basic PyTorch knowledge. You can review your notes from lab and this [PyTorch tutorial](#). The "Introduction to PyTorch" section on the PyTorch website is also helpful.

```
In [ ]: # Import libraries
        from __future__ import print_function
        %matplotlib inline
        import matplotlib
        import matplotlib.pyplot as plt
        import numpy as np
        import torch
```

```
In [ ]: # Fix random seeds so results are reproducible
        torch.manual_seed(0)
        np.random.seed(0)
```

Let's create `torch.tensor` objects for representing the data matrix `X` with targets `Y_or` (for the logical OR function). Each row of `X` is a different input pattern.

```
In [ ]: X_list = [[0.,0.], [0.,1.], [1.,0.], [1.,1.]]
        X = torch.tensor(X_list)
        Y_or = torch.tensor([0.,1.,1.,1.])
        N = X.shape[0] # number of input patterns
        print("Input tensor X:")
        print('  has shape',X.shape)
        print('  and contains',X)
        print('Target tensor Y:')
        print('  has shape',Y_or.shape)
        print('  and contains',Y_or)
```

```
Input tensor X:
  has shape torch.Size([4, 2])
  and contains tensor([[0., 0.],
                      [0., 1.],
                      [1., 0.],
                      [1., 1.]])
Target tensor Y:
  has shape torch.Size([4])
  and contains tensor([0., 1., 1., 1.] )
```

The artificial neuron operates as follows. Given an input vector x (which is one row of input tensor X), the net input (**net**) to the neuron is computed as follows

$$\mathbf{net} = \sum_i x_i w_i + b,$$

for weights w_i and bias b . The activation function $g(\mathbf{net})$ is the logistic function

$$g(\mathbf{net}) = \frac{1}{1 + e^{-\mathbf{net}}},$$

which is used to compute the predicted output $\hat{y} = g(\mathbf{net})$. Finally, the loss (squared error) for a particular pattern x is defined as

$$E(w, b) = (\hat{y} - y)^2,$$

where the target output is y . **Your main task is to manually compute the partial derivatives of the loss E with respect to the neuron parameters:**

$$\frac{\partial E(w, b)}{\partial w}, \frac{\partial E(w, b)}{\partial b}.$$

By manually, we mean to program the gradient computation directly, using the formulas discussed in class. This is in contrast to using PyTorch's `autograd` (Automatic differentiation) that computes the gradient automatically, as discussed in class, lab, and in the PyTorch tutorial (e.g., `loss.backward()`). First, let's write the activation function and the loss in PyTorch.

```
In [ ]: def g_logistic(net):  
        return 1. / (1.+torch.exp(-net))  
  
def loss(yhat,y):  
    return (yhat-y)**2
```

Next, we'll also write two functions for examining the internal operations of the neuron, and the gradients of its parameters.

```
In [ ]: def print_forward(x,yhat,y):  
        # Examine network's prediction for input x  
        print(' Input: ',end='')  
        print(x.numpy())  
        print(' Output: ' + str(round(yhat.item(),3)))  
        print(' Target: ' + str(y.item()))  
  
def print_grad(grad_w,grad_b):  
    # Examine gradients  
    print(' d_loss / d_w = ',end='')  
    print(grad_w)  
    print(' d_loss / d_b = ',end='')  
    print(grad_b)
```

Now let's dive in and begin the implementation of stochastic gradient descent. We'll initialize our parameters w and b randomly, and proceed through a series of epochs of training. Each epoch involves visiting the four training patterns in random order, and updating the parameters after each presentation of an input pattern.

Problem 1 (10 points)

In the code below, fill in code to manually compute the gradient in closed form.

- See lecture slides for the equation for the gradient for the weights w .
- Derive (or reason) to get the equation for the gradient for bias b .

Problem 2 (5 points)

In the code below, fill in code for the weight and bias update rule for gradient descent.

After completing the code, run it to compare **your gradients** with the **ground-truth computed by PyTorch**. (There may be small differences that you shouldn't worry about, e.g. within $1e-6$). Also, you can check the neuron's performance at the end of training.

```
In [ ]: # Initialize parameters
#       Although you will implement gradient descent manually, let's set
#       requires_grad=True
#       anyway so PyTorch will track the gradient too, and we can compare
#       your gradient with PyTorch's.
w = torch.randn(2, requires_grad=True) # [size 2] tensor
b = torch.randn(1, requires_grad=True) # [size 1] tensor

alpha = 0.05 # learning rate
nepochs = 5000 # number of epochs

track_error = []
verbose = True
for e in range(nepochs): # for each epoch
    error_epoch = 0. # sum loss across the epoch
    perm = np.random.permutation(N)
    for p in perm: # visit data points in random order
        x_pat = X[p,:] # get one input pattern

        # compute output of neuron
        net = torch.dot(x_pat,w)+b
        yhat = g_logistic(net)

        # compute loss
        y = Y_or[p]
        myloss = loss(yhat,y)
        error_epoch += myloss.item()

    # Compute the gradient manually
    if verbose:
        print('Compute the gradient manually')
        print_forward(x_pat,yhat,y)
    with torch.no_grad():
        # TODO : YOUR GRADIENT CODE GOES HERE
        # two lines of the form
        #   w_grad = ...      ([size 2] PyTorch tensor)
        #   b_grad = ...      ([size 1] PyTorch tensor)
        # make sure to enclose your code in the "with torch.no_grad()"
        wrapper,
        # otherwise PyTorch will try to track the "gradient" of the
        # gradient computation, which we don't want.
        w_grad = 2*(yhat - y)*yhat*(1 - yhat) * x_pat # dE/dyhat *
```

```

dyhat/dnet * dnet/dw
    b_grad = 2*(yhat - y)*yhat*(1 - yhat)          # dE/dyhat *
dyhat/dnet * dnet/db (=1)
    if verbose: print_grad(w_grad.numpy(),b_grad.numpy())

    # Compute the gradient with PyTorch and compare with manual values
    if verbose: print('Compute the gradient using PyTorch .backward()')
    myloss.backward()
    if verbose:
        print_grad(w.grad.numpy(),b.grad.numpy())
        print("")
    w.grad.zero_() # clear PyTorch's gradient
    b.grad.zero_()

    # Parameter update with gradient descent
    with torch.no_grad():
        # TODO : YOUR PARAMETER UPDATE CODE GOES HERE
        # two lines of the form:
        #     w -= ....
        #     b -= ....
        w -= alpha * w_grad
        b -= alpha * b_grad

    if verbose==True: verbose=False
    track_error.append(error_epoch)
    if e % 50 == 0:
        print("epoch " + str(e) + "; error=" +str(round(error_epoch,3)))

# print a final pass through patterns
for p in range(X.shape[0]):
    x_pat = X[p]
    net = torch.dot(x_pat,w)+b
    yhat = g_logistic(net)
    y = Y_or[p]
    print("Final result:")
    print_forward(x_pat,yhat,y)
    print("")

# track output of gradient descent
plt.figure()
plt.clf()
plt.plot(track_error)
plt.title('stochastic gradient descent (logistic activation)')
plt.ylabel('error for epoch')
plt.xlabel('epoch')
plt.show()

```

```
Compute the gradient manually
Input: [1. 0.]
Output: 0.346
Target: 1.0
d_loss / d_w = [-0.29599196 -0.          ]
d_loss / d_b = [-0.29599196]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [-0.29599196 -0.          ]
d_loss / d_b = [-0.29599196]
```

```
Compute the gradient manually
Input: [1. 1.]
Output: 0.289
Target: 1.0
d_loss / d_w = [-0.29213744 -0.29213744]
d_loss / d_b = [-0.29213744]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [-0.29213747 -0.29213747]
d_loss / d_b = [-0.29213747]
```

```
Compute the gradient manually
Input: [0. 1.]
Output: 0.081
Target: 1.0
d_loss / d_w = [-0.          -0.13688438]
d_loss / d_b = [-0.13688438]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [ 0.          -0.13688436]
d_loss / d_b = [-0.13688436]
```

```
Compute the gradient manually
Input: [0. 0.]
Output: 0.105
Target: 0.0
d_loss / d_w = [0. 0.]
d_loss / d_b = [0.01974555]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [0. 0.]
d_loss / d_b = [0.01974555]
```

```
epoch 0; error=1.789
epoch 50; error=0.456
epoch 100; error=0.267
epoch 150; error=0.218
epoch 200; error=0.188
epoch 250; error=0.166
epoch 300; error=0.149
epoch 350; error=0.135
epoch 400; error=0.123
epoch 450; error=0.113
epoch 500; error=0.104
epoch 550; error=0.097
epoch 600; error=0.09
```

epoch 650; error=0.085
epoch 700; error=0.079
epoch 750; error=0.075
epoch 800; error=0.071
epoch 850; error=0.067
epoch 900; error=0.064
epoch 950; error=0.061
epoch 1000; error=0.058
epoch 1050; error=0.055
epoch 1100; error=0.053
epoch 1150; error=0.051
epoch 1200; error=0.049
epoch 1250; error=0.047
epoch 1300; error=0.045
epoch 1350; error=0.043
epoch 1400; error=0.042
epoch 1450; error=0.04
epoch 1500; error=0.039
epoch 1550; error=0.038
epoch 1600; error=0.037
epoch 1650; error=0.035
epoch 1700; error=0.034
epoch 1750; error=0.033
epoch 1800; error=0.032
epoch 1850; error=0.032
epoch 1900; error=0.031
epoch 1950; error=0.03
epoch 2000; error=0.029
epoch 2050; error=0.028
epoch 2100; error=0.028
epoch 2150; error=0.027
epoch 2200; error=0.026
epoch 2250; error=0.026
epoch 2300; error=0.025
epoch 2350; error=0.025
epoch 2400; error=0.024
epoch 2450; error=0.024
epoch 2500; error=0.023
epoch 2550; error=0.023
epoch 2600; error=0.022
epoch 2650; error=0.022
epoch 2700; error=0.021
epoch 2750; error=0.021
epoch 2800; error=0.02

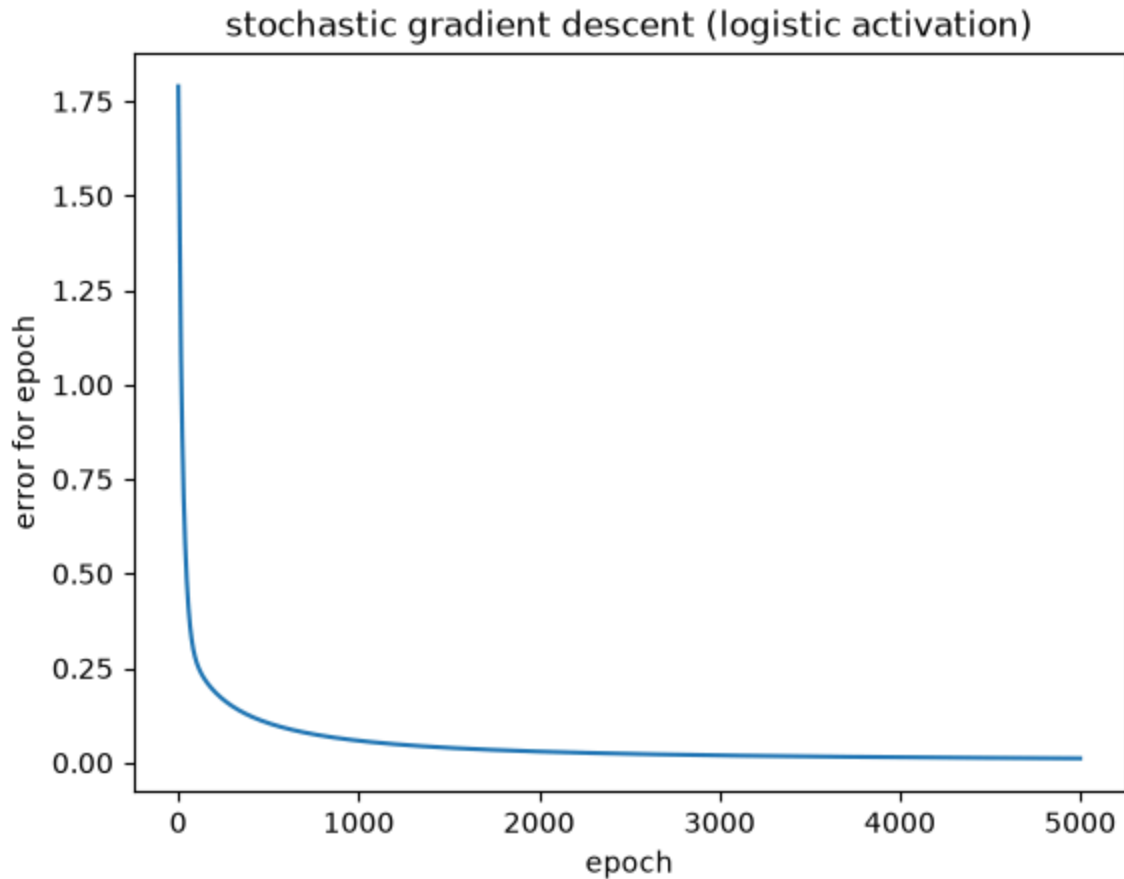
epoch 2850; error=0.02
epoch 2900; error=0.02
epoch 2950; error=0.019
epoch 3000; error=0.019
epoch 3050; error=0.019
epoch 3100; error=0.018
epoch 3150; error=0.018
epoch 3200; error=0.018
epoch 3250; error=0.017
epoch 3300; error=0.017
epoch 3350; error=0.017
epoch 3400; error=0.017
epoch 3450; error=0.016
epoch 3500; error=0.016
epoch 3550; error=0.016
epoch 3600; error=0.016
epoch 3650; error=0.015
epoch 3700; error=0.015
epoch 3750; error=0.015
epoch 3800; error=0.015
epoch 3850; error=0.015
epoch 3900; error=0.014
epoch 3950; error=0.014
epoch 4000; error=0.014
epoch 4050; error=0.014
epoch 4100; error=0.014
epoch 4150; error=0.013
epoch 4200; error=0.013
epoch 4250; error=0.013
epoch 4300; error=0.013
epoch 4350; error=0.013
epoch 4400; error=0.013

epoch 4450; error=0.013
epoch 4500; error=0.012
epoch 4550; error=0.012
epoch 4600; error=0.012
epoch 4650; error=0.012
epoch 4700; error=0.012
epoch 4750; error=0.012
epoch 4800; error=0.012
epoch 4850; error=0.011
epoch 4900; error=0.011
epoch 4950; error=0.011
Final result:
Input: [0. 0.]
Output: 0.079
Target: 0.0

Final result:
Input: [0. 1.]
Output: 0.951
Target: 1.0

Final result:
Input: [1. 0.]
Output: 0.951
Target: 1.0

Final result:
Input: [1. 1.]
Output: 1.0
Target: 1.0



Now let's change the activation function to "linear" (identity function) from the "logistic" function, such that $g(\mathbf{net}) = \mathbf{net}$. With a linear rather than logistic activation, the output will no longer be constrained between 0 and 1. The artificial neuron will still try to solve the problem with 0/1 targets. Here is the simple implementation of $g(\cdot)$:

```
In [ ]: def g_linear(x):
        return x
```

Problem 3 (5 points)

Just as before, fill in the missing code fragments for implementing gradient descent. This time we are using the linear activation function. Be sure to change your gradient calculation to reflect the new activation function.

```
In [ ]: # Initialize parameters
        # Although you will implement gradient descent manually, let's set
        requires_grad=True
        # anyway so PyTorch will track the gradient too, and we can compare
        your gradient with PyTorch's.
        w = torch.randn(2, requires_grad=True) # [size 2] tensor
        b = torch.randn(1, requires_grad=True) # [size 1] tensor
```

```

alpha = 0.05 # learning rate
nepochs = 5000 # number of epochs

track_error = []
verbose = True
for e in range(nepochs): # for each epoch
    error_epoch = 0. # sum loss across the epoch
    perm = np.random.permutation(N)
    for p in perm: # visit data points in random order
        x_pat = X[p,:] # get one input pattern

        # compute output of neuron
        net = torch.dot(x_pat,w)+b
        yhat = g_linear(net)

        # compute loss
        y = Y_or[p]
        myloss = loss(yhat,y)
        error_epoch += myloss.item()

        # Compute the gradient manually
        if verbose:
            print('Compute the gradient manually')
            print_forward(x_pat,yhat,y)
        with torch.no_grad():
            # TODO : YOUR GRADIENT CODE GOES HERE
            # two lines of the form
            # w_grad = ... ([size 2] PyTorch tensor)
            # b_grad = ... ([size 1] PyTorch tensor)
            # make sure to enclose your code in the "with torch.no_grad()"
            wrapper,
            # otherwise PyTorch will try to track the "gradient" of the
            # gradient computation, which we don't want.
            w_grad = 2*(yhat - y) * x_pat # dE/dyhat * dyhat/dnet (=1 for
            # linear) * dnet/dw
            b_grad = 2*(yhat - y) # dE/dyhat * dyhat/dnet (=1 for
            # linear) * dnet/db (=1)
            if verbose: print_grad(w_grad.numpy(),b_grad.numpy())

        # Compute the gradient with PyTorch and compare with manual values
        if verbose: print('Compute the gradient using PyTorch .backward()')
        myloss.backward()
        if verbose:
            print_grad(w.grad.numpy(),b.grad.numpy())
            print('')
        w.grad.zero_() # clear PyTorch's gradient
        b.grad.zero_()

        # Parameter update with gradient descent
        with torch.no_grad():
            # TODO : YOUR PARAMETER UPDATE CODE GOES HERE
            # two lines of the form:
            # w -= ....
            # b -= ....
            w -= alpha * w_grad
            b -= alpha * b_grad

```

```

    if verbose==True: verbose=False
    track_error.append(error_epoch)
    if e % 50 == 0:
        print("epoch " + str(e) + "; error=" +str(round(error_epoch,3)))

# print a final pass through patterns
for p in range(X.shape[0]):
    x_pat = X[p]
    net = torch.dot(x_pat,w)+b
    yhat = g_linear(net)
    y = Y_or[p]
    print("Final result:")
    print_forward(x_pat,yhat,y)
    print("")

# track output of gradient descent
plt.figure()
plt.clf()
plt.plot(track_error)
plt.title('stochastic gradient descent (linear/null activation)')
plt.ylabel('error for epoch')
plt.xlabel('epoch')
plt.show()

```

```
Compute the gradient manually
Input: [1. 1.]
Output: -1.915
Target: 1.0
d_loss / d_w = [-5.8293734 -5.8293734]
d_loss / d_b = [-5.8293734]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [-5.8293734 -5.8293734]
d_loss / d_b = [-5.8293734]
```

```
Compute the gradient manually
Input: [0. 0.]
Output: -1.107
Target: 0.0
d_loss / d_w = [-0. -0.]
d_loss / d_b = [-2.2142534]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [0. 0.]
d_loss / d_b = [-2.2142534]
```

```
Compute the gradient manually
Input: [0. 1.]
Output: -1.789
Target: 1.0
d_loss / d_w = [-0. -5.5789356]
d_loss / d_b = [-5.5789356]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [ 0. -5.5789356]
d_loss / d_b = [-5.5789356]
```

```
Compute the gradient manually
Input: [1. 0.]
Output: 0.142
Target: 1.0
d_loss / d_w = [-1.7151347 -0. ]
d_loss / d_b = [-1.7151347]
Compute the gradient using PyTorch .backward()
d_loss / d_w = [-1.7151347 0. ]
d_loss / d_b = [-1.7151347]
```

```
epoch 0; error=18.238
epoch 50; error=0.3
epoch 100; error=0.298
epoch 150; error=0.299
epoch 200; error=0.297
epoch 250; error=0.304
epoch 300; error=0.304
epoch 350; error=0.306
epoch 400; error=0.307
epoch 450; error=0.309
epoch 500; error=0.308
epoch 550; error=0.3
epoch 600; error=0.305
epoch 650; error=0.3
epoch 700; error=0.308
epoch 750; error=0.3
```

epoch 800; error=0.299
epoch 850; error=0.296
epoch 900; error=0.297
epoch 950; error=0.308
epoch 1000; error=0.301
epoch 1050; error=0.309
epoch 1100; error=0.301
epoch 1150; error=0.308
epoch 1200; error=0.308
epoch 1250; error=0.308
epoch 1300; error=0.307
epoch 1350; error=0.3
epoch 1400; error=0.308
epoch 1450; error=0.306
epoch 1500; error=0.307
epoch 1550; error=0.299
epoch 1600; error=0.307
epoch 1650; error=0.308
epoch 1700; error=0.301
epoch 1750; error=0.308
epoch 1800; error=0.307
epoch 1850; error=0.305
epoch 1900; error=0.306
epoch 1950; error=0.304
epoch 2000; error=0.308
epoch 2050; error=0.304
epoch 2100; error=0.303
epoch 2150; error=0.306
epoch 2200; error=0.304
epoch 2250; error=0.3
epoch 2300; error=0.297
epoch 2350; error=0.301
epoch 2400; error=0.306

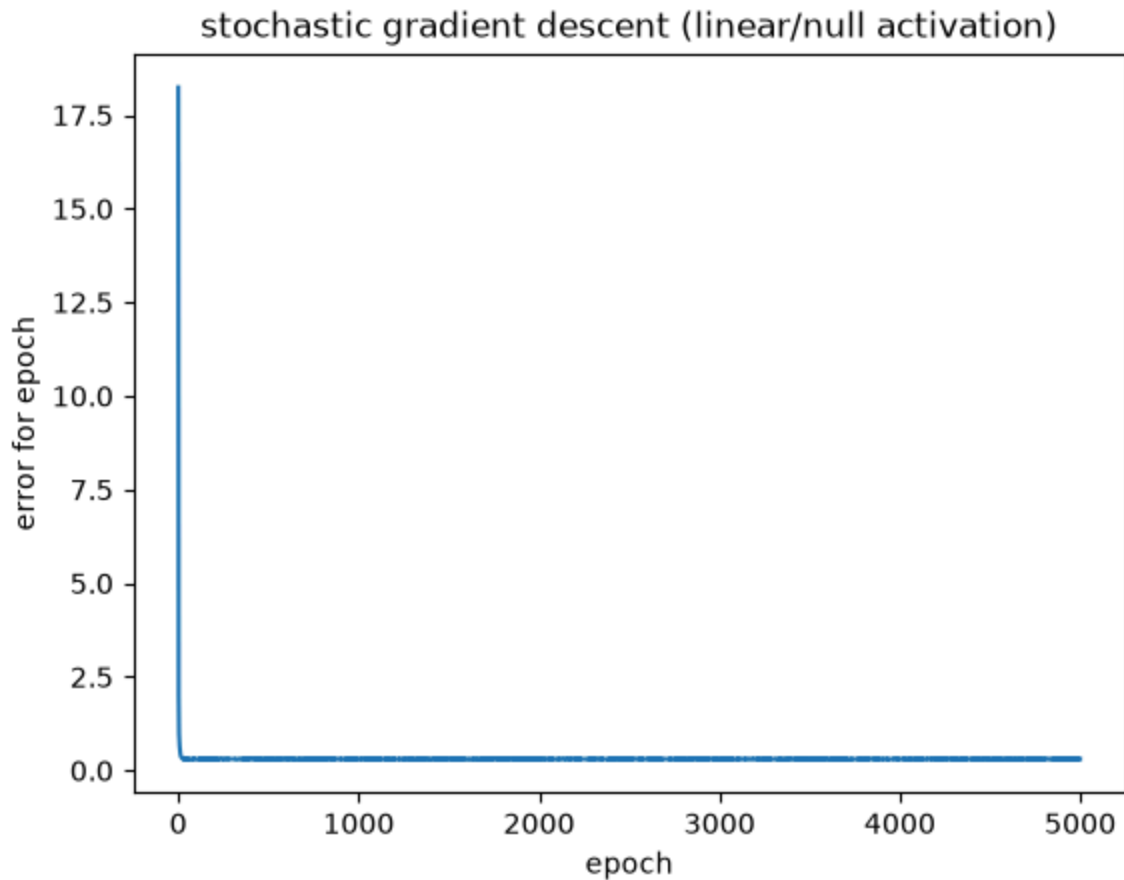
epoch 2450; error=0.308
epoch 2500; error=0.307
epoch 2550; error=0.305
epoch 2600; error=0.301
epoch 2650; error=0.303
epoch 2700; error=0.306
epoch 2750; error=0.3
epoch 2800; error=0.309
epoch 2850; error=0.302
epoch 2900; error=0.308
epoch 2950; error=0.306
epoch 3000; error=0.306
epoch 3050; error=0.303
epoch 3100; error=0.302
epoch 3150; error=0.296
epoch 3200; error=0.303
epoch 3250; error=0.3
epoch 3300; error=0.306
epoch 3350; error=0.303
epoch 3400; error=0.308
epoch 3450; error=0.3
epoch 3500; error=0.306
epoch 3550; error=0.303
epoch 3600; error=0.306
epoch 3650; error=0.307
epoch 3700; error=0.296
epoch 3750; error=0.308
epoch 3800; error=0.308
epoch 3850; error=0.306
epoch 3900; error=0.303
epoch 3950; error=0.31
epoch 4000; error=0.308
epoch 4050; error=0.298
epoch 4100; error=0.304
epoch 4150; error=0.308
epoch 4200; error=0.305
epoch 4250; error=0.306
epoch 4300; error=0.307
epoch 4350; error=0.306
epoch 4400; error=0.31
epoch 4450; error=0.307
epoch 4500; error=0.3
epoch 4550; error=0.309
epoch 4600; error=0.3
epoch 4650; error=0.304
epoch 4700; error=0.306
epoch 4750; error=0.307

```
epoch 4800; error=0.306
epoch 4850; error=0.303
epoch 4900; error=0.31
epoch 4950; error=0.301
Final result:
  Input: [0. 0.]
  Output: 0.267
  Target: 0.0
```

```
Final result:
  Input: [0. 1.]
  Output: 0.748
  Target: 1.0
```

```
Final result:
  Input: [1. 0.]
  Output: 0.751
  Target: 1.0
```

```
Final result:
  Input: [1. 1.]
  Output: 1.231
  Target: 1.0
```



```
In [ ]: # Learned parameters for the linear neuron
print('w =',w.data.numpy(), ' b =',b.item())
```

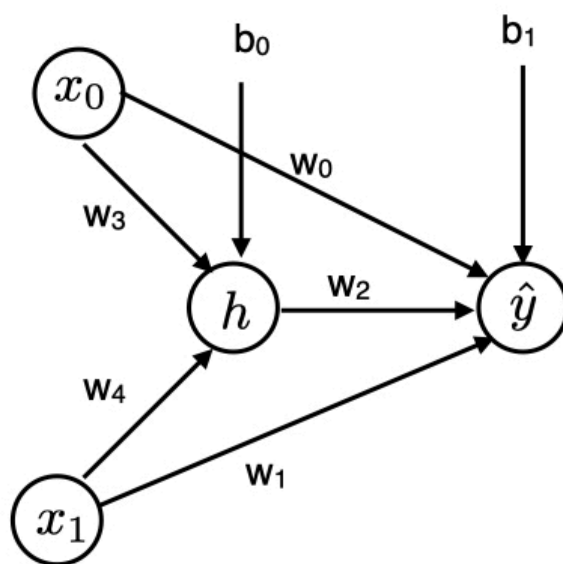
```
w = [0.48346823 0.4808231 ]  b = 0.2670857012271881
```

Problem 4 (10 points)

You'll see above that the artificial neuron, with the simple linear (identity) activation, does worse on the OR problem. Examine the learned weights and bias, and explain why the network does not arrive at a perfect solution.

Answer: The learned parameters are $w_0 \approx 0.48$, $w_1 \approx 0.48$, $b \approx 0.27$, giving outputs of about 0.27, 0.75, 0.75, and 1.23 for the four patterns (targets 0, 1, 1, 1). With a linear activation, the output is just $w_0x_0 + w_1x_1 + b$, so each active input adds a fixed amount and the effects simply sum; there is no squashing function to "saturate" at 1. OR needs the output to be the same (1) whether one or both inputs are on, but a linear unit cannot do this: if the weights were large enough (≈ 1) to bring $[0,1]$ and $[1,0]$ up to 1, then $[1,1]$ would output ≈ 2 , and the bias needed for $[0,0]$ to be 0 would also be in conflict. Since no linear function fits all four targets exactly, gradient descent instead finds the compromise that minimizes the total squared error, which is $w_0 = w_1 = 0.5$, $b = 0.25$ (outputs 0.25, 0.75, 0.75, 1.25, each off by 0.25). Our learned values hover around this least-squares solution because stochastic updates keep nudging them. The logistic neuron avoids this problem because its output flattens out near 1, so a large net input from one or two active inputs produces nearly the same output.

In the next part, we have a simple multi-layer network with two input neurons, one hidden neuron, and one output neuron. Both the hidden and output unit should use the logistic activation function. We will learn to compute logical XOR. The network and logical XOR are shown below, for inputs x_0 and x_1 and target output y .



logical XOR

x_0	x_1	y
0	0	0
0	1	1
1	0	1
1	1	0

Problem 5 (15 points)

You will implement backpropagation for this simple network. In the code below, you have several parts to fill in. First, define the forward pass to compute the output $\hat{y}$ from the input x . Second, fill in code to manually compute the gradients for all five weights w and two biases b in closed form. Third, fill in the code for updating the biases and weights.

After completing the code, run it to compare **your gradients** with the **ground-truth computed by PyTorch**. (There may be small differences that you shouldn't worry about, e.g. within $1e-6$). Also, you can check the network's performance at the end of training.

```
In [ ]: # Same input tensor X and new labels y for xor
Y_xor = torch.tensor([0.,1.,1.,0.])
N = X.shape[0] # number of input patterns

# Initialize parameters
# Although you will implement gradient descent manually, let's set
requires_grad=True
# anyway so PyTorch will track the gradient too, and we can compare
your gradient with PyTorch's.
w_34 = torch.randn(2,requires_grad=True) # [size 2] tensor representing
[w_3,w_4]
w_012 = torch.randn(3,requires_grad=True) # [size 3] tensor representing
[w_0,w_1,w_2]
b_0 = torch.randn(1,requires_grad=True) # [size 1] tensor
b_1 = torch.randn(1,requires_grad=True) # [size 1] tensor

alpha = 0.05 # learning rate
nepochs = 10000 # number of epochs

track_error = []
verbose = True
for e in range(nepochs): # for each epoch
    error_epoch = 0. # sum loss across the epoch
    perm = np.random.permutation(N)
    for p in perm: # visit data points in random order
        x_pat = X[p,:] # input pattern

        # Compute the output of hidden neuron h
        # e.g., two lines like the following
        # net_h = ...
        # h = ...
        # TODO : YOUR CODE GOES HERE
        net_h = torch.dot(x_pat,w_34) + b_0
        h = g_logistic(net_h)

        # Compute the output of neuron yhat
        # e.g., two lines like the following
        # net_y = ...
        # yhat = ...
        # TODO : YOUR CODE GOES HERE
        net_y = torch.dot(x_pat,w_012[:2]) + w_012[2]*h + b_1
```

```

yhat = g_logistic(net_y)

# compute loss
y = Y_xor[p]
myloss = loss(yhat,y)
error_epoch += myloss.item()

# print output if this is the last epoch
if (e == nepochs-1):
    print("Final result:")
    print_forward(x_pat,yhat,y)
    print("")

# Compute the gradient manually
if verbose:
    print('Compute the gradient manually')
    print_forward(x_pat,yhat,y)
with torch.no_grad():
    # TODO : YOUR GRADIENT CODE GOES HERE
    # should include at least these 4 lines (helper lines may be
useful)
    #     w_34_grad = ...
    #     b_0_grad = ...
    #     w_012_grad = ...
    #     b_1_grad = ...
    # make sure to enclose your code in the "with torch.no_grad()"
wrapper,
    # otherwise PyTorch will try to track the "gradient" of the
gradient computation, which we don't want.
    delta_y = 2*(yhat - y)*yhat*(1 - yhat) # error signal at output
    w_012_grad = delta_y * torch.cat((x_pat,h)) # inputs to output
unit: [x0,x1,h]
    b_1_grad = delta_y
    delta_h = delta_y * w_012[2] * h*(1 - h) # backpropagate
through w_2
    w_34_grad = delta_h * x_pat
    b_0_grad = delta_h
if verbose:
    print(" Grad for w_34 and b_0")
    print_grad(w_34_grad.numpy(),b_0_grad.numpy())
    print(" Grad for w_012 and b_1")
    print_grad(w_012_grad.numpy(),b_1_grad.numpy())
    print("")

# Compute the gradient with PyTorch and compare with manual values
if verbose: print('Compute the gradient using PyTorch .backward()')
myloss.backward()
if verbose:
    print(" Grad for w_34 and b_0")
    print_grad(w_34.grad.numpy(),b_0.grad.numpy())
    print(" Grad for w_012 and b_1")
    print_grad(w_012.grad.numpy(),b_1.grad.numpy())
    print("")
w_34.grad.zero_() # clear PyTorch's gradient
b_0.grad.zero_()
w_012.grad.zero_()

```

```

b_1.grad.zero_()

# Parameter update with gradient descent
with torch.no_grad():
    # TODO : YOUR PARAMETER UPDATE CODE GOES HERE
    # Four lines of the form
    # w_34 -= ...
    # b_0 -= ...
    # w_012 -= ...
    # b_1 -= ...
    w_34 -= alpha * w_34_grad
    b_0 -= alpha * b_0_grad
    w_012 -= alpha * w_012_grad
    b_1 -= alpha * b_1_grad

if verbose==True: verbose=False
track_error.append(error_epoch)
if e % 50 == 0:
    print("epoch " + str(e) + "; error=" +str(round(error_epoch,3)))

# track output of gradient descent
plt.figure()
plt.clf()
plt.plot(track_error)
plt.title('stochastic gradient descent (XOR)')
plt.ylabel('error for epoch')
plt.xlabel('epoch')
plt.show()

```

Compute the gradient manually

Input: [1. 0.]

Output: 0.124

Target: 1.0

Grad for w_34 and b_0

d_loss / d_w = [0.02602027 0.]

d_loss / d_b = [0.02602027]

Grad for w_012 and b_1

d_loss / d_w = [-0.18982297 -0. -0.12192427]

d_loss / d_b = [-0.18982297]

Compute the gradient using PyTorch .backward()

Grad for w_34 and b_0

d_loss / d_w = [0.02602027 0.]

d_loss / d_b = [0.02602027]

Grad for w_012 and b_1

d_loss / d_w = [-0.18982297 0. -0.12192427]

d_loss / d_b = [-0.18982297]

Compute the gradient manually

Input: [0. 0.]

Output: 0.237

Target: 0.0

Grad for w_34 and b_0

d_loss / d_w = [-0. -0.]

d_loss / d_b = [-0.01255374]

Grad for w_012 and b_1

d_loss / d_w = [0. 0. 0.04672741]

d_loss / d_b = [0.08572872]

Compute the gradient using PyTorch .backward()

Grad for w_34 and b_0

d_loss / d_w = [0. 0.]

d_loss / d_b = [-0.01255374]

Grad for w_012 and b_1

d_loss / d_w = [0. 0. 0.04672741]

d_loss / d_b = [0.08572872]

Compute the gradient manually

Input: [0. 1.]

Output: 0.156

Target: 1.0

Grad for w_34 and b_0

d_loss / d_w = [0. 0.02564379]

d_loss / d_b = [0.02564379]

Grad for w_012 and b_1

d_loss / d_w = [-0. -0.22198938 -0.16312972]

d_loss / d_b = [-0.22198938]

Compute the gradient using PyTorch .backward()

Grad for w_34 and b_0

d_loss / d_w = [0. 0.02564379]

d_loss / d_b = [0.02564379]

Grad for w_012 and b_1

d_loss / d_w = [0. -0.2219894 -0.16312972]

d_loss / d_b = [-0.2219894]

Compute the gradient manually

Input: [1. 1.]

Output: 0.082

Target: 0.0

Grad for w_34 and b_0

d_loss / d_w = [-0.001137 -0.001137]

d_loss / d_b = [-0.001137]

Grad for w_012 and b_1

d_loss / d_w = [0.01239525 0.01239525 0.00998017]

d_loss / d_b = [0.01239525]

Compute the gradient using PyTorch .backward()

Grad for w_34 and b_0

d_loss / d_w = [-0.001137 -0.001137]

d_loss / d_b = [-0.001137]

Grad for w_012 and b_1

d_loss / d_w = [0.01239525 0.01239525 0.00998017]

d_loss / d_b = [0.01239525]

epoch 0; error=1.544

epoch 50; error=1.039

epoch 100; error=1.017

epoch 150; error=1.016

epoch 200; error=1.016

epoch 250; error=1.016

epoch 300; error=1.015

epoch 350; error=1.015

epoch 400; error=1.015

epoch 450; error=1.015

epoch 500; error=1.015

epoch 550; error=1.015

epoch 600; error=1.015

epoch 650; error=1.015

epoch 700; error=1.015

epoch 750; error=1.014

epoch 800; error=1.014

epoch 850; error=1.014

epoch 900; error=1.014

epoch 950; error=1.014

epoch 1000; error=1.014

epoch 1050; error=1.014

epoch 1100; error=1.014

epoch 1150; error=1.014

epoch 1200; error=1.013

epoch 1250; error=1.013

epoch 1300; error=1.013

epoch 1350; error=1.013

epoch 1400; error=1.013

epoch 1450; error=1.013

epoch 1500; error=1.012

epoch 1550; error=1.012

epoch 1600; error=1.011

epoch 1650; error=1.011

epoch 1700; error=1.01

epoch 1750; error=1.01

epoch 1800; error=1.009
epoch 1850; error=1.008
epoch 1900; error=1.007
epoch 1950; error=1.006
epoch 2000; error=1.005
epoch 2050; error=1.004
epoch 2100; error=1.002
epoch 2150; error=1.0
epoch 2200; error=0.998
epoch 2250; error=0.995
epoch 2300; error=0.992
epoch 2350; error=0.989
epoch 2400; error=0.984
epoch 2450; error=0.98
epoch 2500; error=0.974
epoch 2550; error=0.968
epoch 2600; error=0.961
epoch 2650; error=0.953
epoch 2700; error=0.944
epoch 2750; error=0.933
epoch 2800; error=0.921
epoch 2850; error=0.907
epoch 2900; error=0.892
epoch 2950; error=0.874
epoch 3000; error=0.855
epoch 3050; error=0.833
epoch 3100; error=0.81
epoch 3150; error=0.785
epoch 3200; error=0.758
epoch 3250; error=0.73
epoch 3300; error=0.7
epoch 3350; error=0.67
epoch 3400; error=0.639
epoch 3450; error=0.609
epoch 3500; error=0.579
epoch 3550; error=0.549
epoch 3600; error=0.52
epoch 3650; error=0.493
epoch 3700; error=0.466
epoch 3750; error=0.441
epoch 3800; error=0.417
epoch 3850; error=0.395
epoch 3900; error=0.374
epoch 3950; error=0.354
epoch 4000; error=0.336
epoch 4050; error=0.318
epoch 4100; error=0.302
epoch 4150; error=0.287
epoch 4200; error=0.273
epoch 4250; error=0.26
epoch 4300; error=0.248
epoch 4350; error=0.237
epoch 4400; error=0.226
epoch 4450; error=0.217
epoch 4500; error=0.207
epoch 4550; error=0.199

epoch 4600; error=0.191
epoch 4650; error=0.183
epoch 4700; error=0.176
epoch 4750; error=0.169
epoch 4800; error=0.163
epoch 4850; error=0.157
epoch 4900; error=0.152
epoch 4950; error=0.147
epoch 5000; error=0.142
epoch 5050; error=0.137
epoch 5100; error=0.133
epoch 5150; error=0.128
epoch 5200; error=0.124
epoch 5250; error=0.121
epoch 5300; error=0.117
epoch 5350; error=0.114
epoch 5400; error=0.111
epoch 5450; error=0.107
epoch 5500; error=0.105
epoch 5550; error=0.102
epoch 5600; error=0.099
epoch 5650; error=0.097
epoch 5700; error=0.094
epoch 5750; error=0.092
epoch 5800; error=0.09
epoch 5850; error=0.088
epoch 5900; error=0.085
epoch 5950; error=0.084
epoch 6000; error=0.082
epoch 6050; error=0.08
epoch 6100; error=0.078
epoch 6150; error=0.076
epoch 6200; error=0.075
epoch 6250; error=0.073
epoch 6300; error=0.072
epoch 6350; error=0.07
epoch 6400; error=0.069
epoch 6450; error=0.068
epoch 6500; error=0.066
epoch 6550; error=0.065
epoch 6600; error=0.064
epoch 6650; error=0.063
epoch 6700; error=0.062
epoch 6750; error=0.061
epoch 6800; error=0.059
epoch 6850; error=0.058
epoch 6900; error=0.057
epoch 6950; error=0.057
epoch 7000; error=0.056
epoch 7050; error=0.055

epoch 7100; error=0.054
epoch 7150; error=0.053
epoch 7200; error=0.052
epoch 7250; error=0.051
epoch 7300; error=0.051
epoch 7350; error=0.05
epoch 7400; error=0.049
epoch 7450; error=0.048
epoch 7500; error=0.048
epoch 7550; error=0.047
epoch 7600; error=0.046
epoch 7650; error=0.046
epoch 7700; error=0.045
epoch 7750; error=0.044
epoch 7800; error=0.044
epoch 7850; error=0.043
epoch 7900; error=0.043
epoch 7950; error=0.042
epoch 8000; error=0.042
epoch 8050; error=0.041
epoch 8100; error=0.04
epoch 8150; error=0.04
epoch 8200; error=0.04
epoch 8250; error=0.039
epoch 8300; error=0.039
epoch 8350; error=0.038
epoch 8400; error=0.038
epoch 8450; error=0.037
epoch 8500; error=0.037
epoch 8550; error=0.036
epoch 8600; error=0.036
epoch 8650; error=0.036
epoch 8700; error=0.035
epoch 8750; error=0.035
epoch 8800; error=0.034
epoch 8850; error=0.034
epoch 8900; error=0.034
epoch 8950; error=0.033
epoch 9000; error=0.033
epoch 9050; error=0.033
epoch 9100; error=0.032
epoch 9150; error=0.032
epoch 9200; error=0.032
epoch 9250; error=0.031
epoch 9300; error=0.031
epoch 9350; error=0.031
epoch 9400; error=0.03
epoch 9450; error=0.03
epoch 9500; error=0.03
epoch 9550; error=0.029
epoch 9600; error=0.029
epoch 9650; error=0.029
epoch 9700; error=0.029
epoch 9750; error=0.028
epoch 9800; error=0.028
epoch 9850; error=0.028

epoch 9900; error=0.028

epoch 9950; error=0.027

Final result:

Input: [1. 0.]

Output: 0.919

Target: 1.0

Final result:

Input: [0. 0.]

Output: 0.073

Target: 0.0

Final result:

Input: [0. 1.]

Output: 0.919

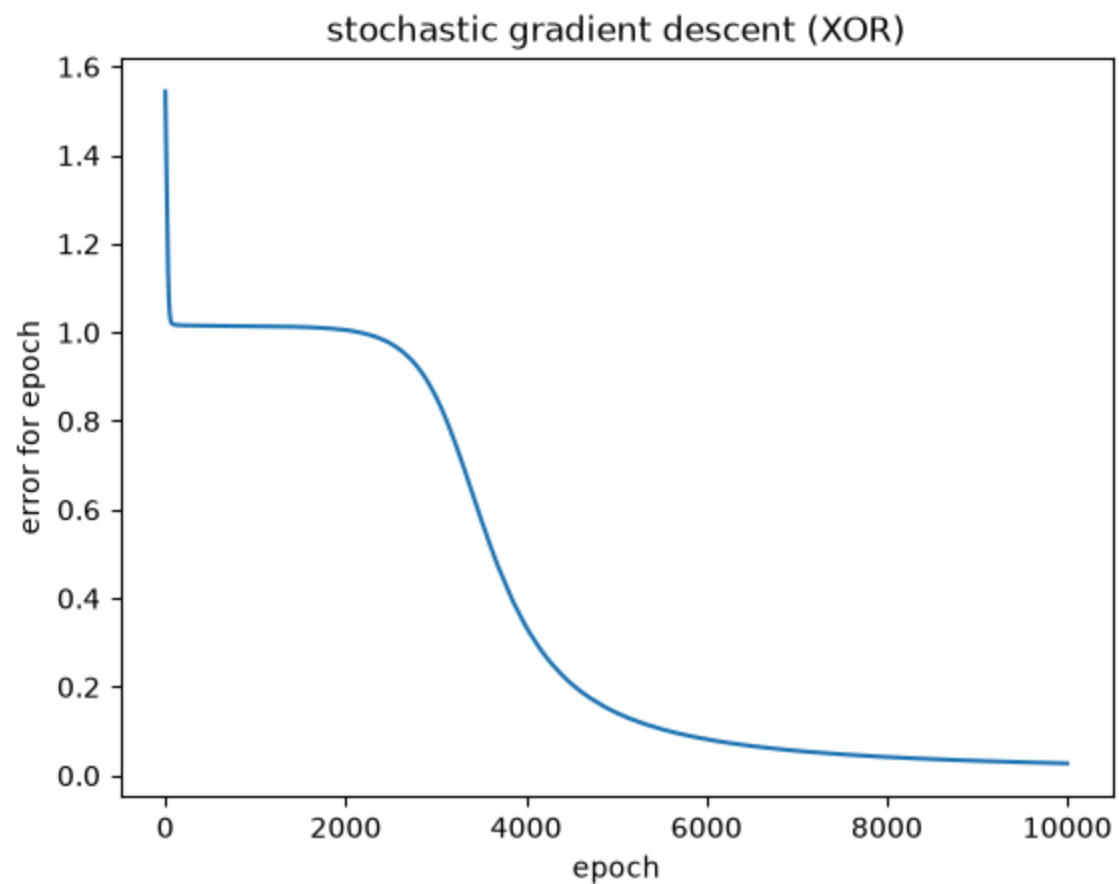
Target: 1.0

Final result:

Input: [1. 1.]

Output: 0.094

Target: 0.0



Problem 6 (10 points)

After running your XOR network, print the values of the learned weights and biases.

Your job now is to describe the solution that the network has learned. How does it work? Walk through each input pattern to describe how the network computes the right answer (if it does). See discussion in lecture for an example.

```
In [ ]: # Learned XOR parameters, and what each unit does for each input pattern
print('w_34 =',w_34.data.numpy(), ' b_0 =',b_0.item())
print('w_012 =',w_012.data.numpy(), ' b_1 =',b_1.item())
with torch.no_grad():
    for p in range(N):
        x_pat = X[p]
        net_h = torch.dot(x_pat,w_34) + b_0
        h = g_logistic(net_h)
        net_y = torch.dot(x_pat,w_012[:2]) + w_012[2]*h + b_1
        yhat = g_logistic(net_y)
        print(f'x={x_pat.numpy()} net_h={net_h.item():7.2f} h={
{h.item():.3f} net_y={net_y.item():7.2f} yhat={yhat.item():.3f} target=
{Y_xor[p].item():.0f}')
```

```
w_34 = [6.6760035 6.677774 ] b_0 = -2.728466749191284
w_012 = [-4.90621 -4.906645 10.74449 ] b_1 = -3.2046456336975098
x=[0. 0.] net_h= -2.73 h=0.061 net_y= -2.55 yhat=0.073 target=0
x=[0. 1.] net_h= 3.95 h=0.981 net_y= 2.43 yhat=0.919 target=1
x=[1. 0.] net_h= 3.95 h=0.981 net_y= 2.43 yhat=0.919 target=1
x=[1. 1.] net_h= 10.63 h=1.000 net_y= -2.27 yhat=0.093 target=0
```

Answer: The learned parameters are $w_3 \approx 6.68$, $w_4 \approx 6.68$, $b_0 \approx -2.73$ (hidden unit) and $w_0 \approx -4.91$, $w_1 \approx -4.91$, $w_2 \approx 10.74$, $b_1 \approx -3.20$ (output unit).

How it works: The hidden unit h acts as an **OR detector**: its bias is negative, but either input alone (weight ≈ 6.7) is enough to overcome it and turn h on. The output unit is strongly excited by h ($w_2 \approx 10.7$) but is also **inhibited directly by each input** ($w_0, w_1 \approx -4.9$). One active input's inhibition is not enough to cancel h 's excitation, but two active inputs together are. So the output computes "OR, but not when both inputs are on", which is XOR.

Walking through each pattern:

- **[0,0]:** $net_h = -2.73$, so $h \approx 0.06$ (off). The output receives almost nothing except its negative bias: $net_y \approx -3.20 + 10.74(0.06) = -2.55$, so $\hat{y} \approx 0.07 \checkmark$ (target 0).
- **[0,1]:** $net_h = 6.68 - 2.73 = 3.95$, so $h \approx 0.98$ (on). At the output, h contributes $+10.54$, the active input x_1 contributes -4.91 , and the bias -3.20 : $net_y \approx 2.43$, so $\hat{y} \approx 0.92 \checkmark$ (target 1).
- **[1,0]:** Identical to [0,1] by symmetry (the two input weights are almost equal): $h \approx 0.98$, $net_y \approx 2.43$, $\hat{y} \approx 0.92 \checkmark$ (target 1).
- **[1,1]:** $net_h = 10.63$, so $h \approx 1.00$ (on). The output gets $+10.74$ from h , but now **both** inputs inhibit it, $-4.91 - 4.91 = -9.81$, plus the bias -3.20 : $net_y \approx -2.27$, so $\hat{y} \approx 0.09 \checkmark$ (target 0).

So the network does compute XOR correctly. The hidden unit solves the part a single neuron can do (OR), and the direct input-to-output connections subtract it back out when both inputs are on. The key is that w_2 is larger than one input's inhibition but smaller than two inputs' inhibition combined ($4.91 < 10.74 - 3.20 < 9.81$).